

Anleitung: Lagebild-Dashboard lokal starten und bauen

Diese Anleitung ist für Einsteiger gedacht. Du brauchst **keine Vorkenntnisse** und musst **Node.js noch nicht installiert haben**. Sie gilt für Windows, macOS und Linux, und alles läuft über das Terminal.

Ergebnis: Das Dashboard läuft auf deinem Rechner und ist im Browser unter `http://localhost:4321/katastrophenschutz--dashboard/` erreichbar.

Dauer: ca. 10–15 Minuten. Du brauchst eine Internetverbindung, weil die Datenquellen (Wetter, Pegel ...) live abgefragt werden.

Schritt 1: Terminal öffnen

Das Terminal ist ein Textfenster, in dem du Befehle eingibst. Alles Weitere geht darüber, du musst nichts von Webseiten herunterladen. Die Befehle einfach kopieren, einfügen und mit Enter bestätigen.

- **Windows:** Startmenü öffnen, PowerShell eintippen und **Windows PowerShell** starten.
- **macOS:** `Cmd + Leertaste`, dann `Terminal` eintippen und Enter.
- **Linux:** `Strg + Alt + T` (oder „Terminal“ im Anwendungsmenü).

Schritt 2: Node.js installieren

Node.js ist die Laufzeitumgebung, die das Projekt zum Bauen und Starten braucht. Version **22.12 oder neuer** ist nötig. Wähle den Abschnitt für dein System.

Windows (PowerShell):

```
winget install OpenJS.NodeJS.LTS
```

Falls nach einer Zustimmung gefragt wird, mit `Y` und Enter bestätigen. `winget` ist ab Windows 10 (Version 1809) bzw. Windows 11 vorhanden.

macOS (Terminal): Zuerst Homebrew installieren (falls noch nicht vorhanden, prüfen mit `brew --version`):

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

Homebrew fragt nach deinem Mac-Passwort (beim Tippen erscheint nichts, das ist normal) und zeigt am Ende ggf. zwei Befehle unter „Next steps“ an, die du ebenfalls ausführen musst. Danach:

```
brew install node
```

Linux (Terminal): Über den Node Version Manager `nvm`, weil die Node-Pakete der Distributionen oft zu alt sind:

```
curl -o- https://raw.githubusercontent.com/nvm-sh/nvm/v0.40.3/install.sh | bash
```

Terminal schließen und neu öffnen, dann:

```
nvm install --lts
```

Für `curl` ggf. vorher `sudo apt install curl` (Debian/Ubuntu) ausführen.

Wichtig (alle Systeme): Schließe das Terminal nach der Installation und öffne ein neues. Sonst kennt es Node noch nicht.

Prüfen, ob es geklappt hat:

```
node --version  
npm --version
```

Bei `node` sollte `v22.12.0` oder höher erscheinen. `npm` (der Paketmanager) wird automatisch mitinstalliert.

Schritt 3: Projekt entpacken

Das Projekt bekommst du als **ZIP-Datei** von deinem Mitschüler, du musst nichts herunterladen. Speichere die Datei irgendwo, z. B. im Ordner „Downloads“, und entpacke sie im Terminal. Ersetze dabei `projekt.zip` durch den echten Dateinamen.

Windows (PowerShell):

```
cd $HOME\Downloads
Expand-Archive projekt.zip .
```

macOS / Linux:

```
cd ~/Downloads
unzip projekt.zip
```

(Falls `unzip` unter Linux fehlt: `sudo apt install unzip`.) Du kannst die ZIP-Datei auch bequem per Doppelklick bzw. Rechtsklick → „Entpacken“ auslösen.

Wechsle danach in den entpackten Ordner. Den Namen siehst du mit `ls` (macOS/Linux) bzw. `dir` (Windows):

```
cd ordnername
```

Prüfen, ob du im richtigen Ordner bist: `ls` bzw. `dir`. Du solltest `package.json`, `README.md` und den Ordner `src` sehen. Liegt in dem Ordner nur ein weiterer Ordner, wechsle mit `cd` noch eine Ebene tiefer.

Schritt 4: Abhängigkeiten installieren (einmalig)

```
npm install
```

Das lädt alle benötigten Bausteine (u. a. das Framework Astro) in den Ordner `node_modules`. Das dauert ein bis zwei Minuten. Es erscheinen viele Zeilen Text, das ist normal. Am Ende darf gern eine Meldung wie „added ... packages“ stehen. Diesen Schritt musst du nur **einmal** machen.

Schritt 5: Dashboard starten

Zum Ausprobieren (Entwicklungsserver):

```
npm run dev
```

Im Terminal erscheint eine Adresse. Öffne diese im Browser:

```
http://localhost:4321/katastrophenschutz-dashboard/
```

Achtung: Den hinteren Teil `/katastrophenschutz-dashboard/` unbedingt mitschreiben. Nur `localhost:4321` zeigt eine Fehlerseite.

Das Terminal-Fenster muss dabei **offen bleiben**. Beenden kannst du den Server mit `Strg + C`.

Als fertigen Build (so wie später im Internet):

```
npm run build
npm run preview
```

- `npm run build` erzeugt die fertige Seite im Ordner `dist`.
 - `npm run preview` startet diese fertige Seite lokal. Die Adresse steht wieder im Terminal, in der Regel dieselbe wie oben.
-

Optional: Beispielwarnung ansehen

Meist gibt es gerade keine echte Warnung. Um die Warnungen-Kachel mit einer Testwarnung zu sehen:

macOS / Linux:

```
DASHBOARD_TEST_WARNING=1 npm run build && npm run preview
```

Windows (Eingabeaufforderung):

```
set DASHBOARD_TEST_WARNING=1 && npm run build && npm run preview
```

Windows (PowerShell):

```
$env:DASHBOARD_TEST_WARNING=1; npm run build; npm run preview
```

Häufige Probleme

Problem	Lösung
node / npm wird nicht gefunden	Node.js ist nicht (richtig) installiert, oder das Terminal wurde vor der Installation geöffnet. Terminal schließen, neu öffnen, erneut node --version prüfen. Sonst Schritt 2 wiederholen.
Windows: „Die Ausführung von Skripts ist auf diesem System deaktiviert“ bei npm	Einmalig in PowerShell Set-ExecutionPolicy -Scope CurrentUser RemoteSigned ausführen, mit J bestätigen, Terminal neu öffnen.
Windows: winget wird nicht gefunden	Windows ist zu alt oder der „App-Installer“ fehlt. Im Microsoft Store den „App-Installer“ aktualisieren, oder Node.js von https://nodejs.org installieren.
Fehlermeldung zur Node-Version (z. B. „requires Node >=22.12.0“)	Deine Node-Version ist zu alt. Installiere Node.js wie in Schritt 2 neu bzw. aktualisiere es (Windows: winget upgrade OpenJS.NodeJS.LTS, macOS: brew upgrade node, Linux: nvm install --lts).
npm install meldet „no such file“ / package.json nicht gefunden	Du bist im falschen Ordner. Wechsle mit cd in den Ordner, in dem package.json liegt (Schritt 3).
Im Browser „404“ oder leere Seite	Die Adresse muss mit /katastrophenschutz-dashboard/ enden.
„Port 4321 is already in use“	Es läuft schon ein Dashboard. Altes Terminal mit Strg + C beenden, dann erneut starten.
Kacheln bleiben leer	Prüfe die Internetverbindung. Manche Schulnetzwerke blockieren einzelne Schnittstellen. Dann fehlen nur diese Kacheln.
Warnungen-Kachel zeigt „keine aktiven Warnungen“	Normal, wenn gerade keine Warnung vorliegt. Zum Testen siehe „Beispielwarnung“.
Nach npm install erscheinen Warnungen (warn, deprecated)	Meist harmlos. Nur Meldungen mit ERR! / error sind Fehler.

Kurz zusammengefasst

```
# einmalig
npm install

# zum Starten
npm run dev
# dann im Browser öffnen: http://localhost:4321/katastrophenschutz-dashboard/
```

Weitere Informationen zum Projekt (Datenquellen, Aufbau, Deployment) stehen in der Datei README.md im Projektordner.